Explainable AI: Concepts, Techniques, and Implementations

From Taxonomy to Practical Methods (LIME, SHAP, Grad-CAM, and Beyond)

Previous session

Guided backpropagation algorithm with mathematical foundation

Today's Session Overview

In today's session, we will explore:

- Grad-CAM
- Class Activation Maps (CAMs)

Introduction

When we apply XAI methods, **the CNN model is already fully trained** (i.e., the convolutional kernels, dense-layer weights, and biases are all optimized).

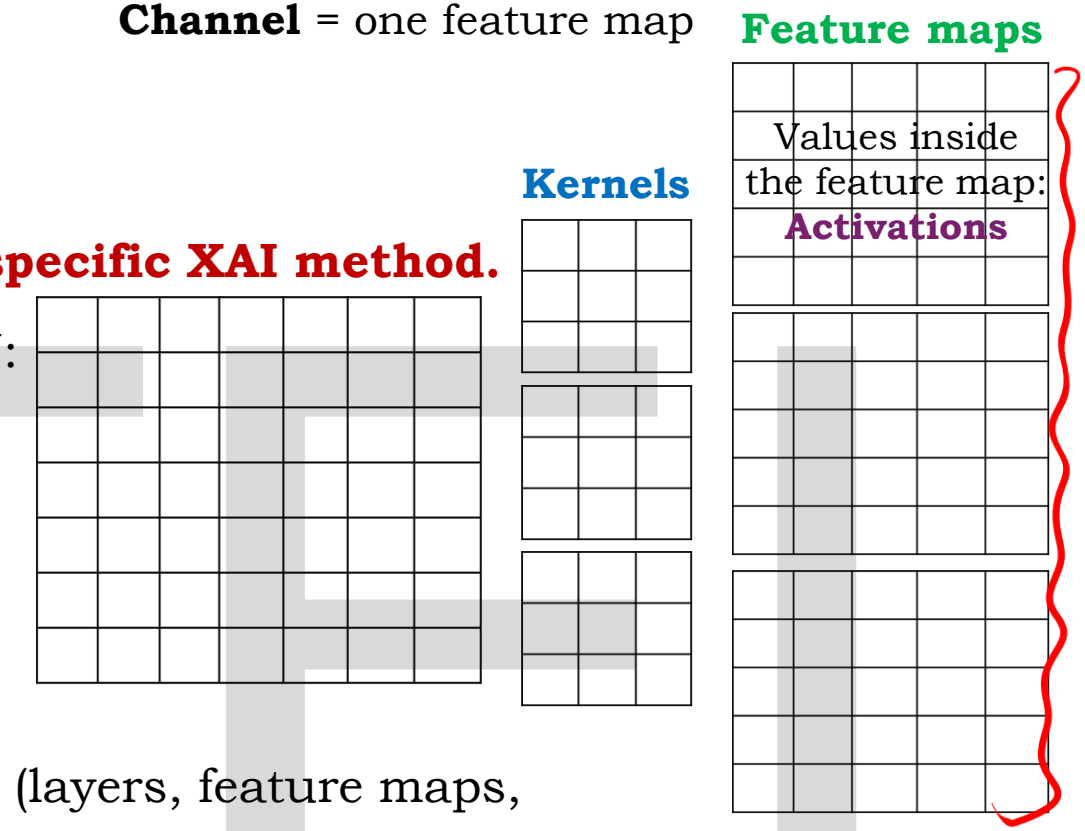
During Testing:

- A new input image is passed to the trained CNN model (forward pass).
- The model produces a prediction (e.g., *cat*, *dog*, *tumor*, etc.).
- Our goal now is to **understand which regions of the input image contributed most to the model's prediction**.

To achieve this, we apply XAI technique: Grad-CAM and CAM.

Grad-CAM

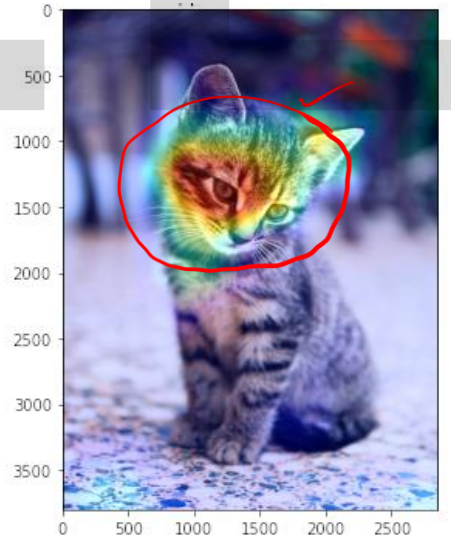
Channel = one feature map



- Gradient-weighted Class Activation Mapping is a **model-specific XAI method.**
- Grad-CAM depends on the internal components of a CNN:
  - ➢**the convolution layer (Last conv layer before flatten)**
  - ➢**the feature maps in those layer**
  - ➢**the channel-wise activations (Post ReLU)**
  - ➢**the gradients flowing through those layer**
- Model requirement:
  - ➢**Grad-CAM works only on CNN based models.**

  Grad-CAM needs access to these **internal components** (layers, feature maps, activations, gradients).

Grad-CAM is an explainability technique that highlights the regions in an image that are most important for a model's classification decision by computing the **gradient of the class logit (before softmax) with respect to the feature maps in the final convolutional layer**, effectively showing **which spatial regions the model focuses on when making a prediction.**



Gradient measures how much the class logit (score for class C) changes when we make a small change in each value of the feature map in the last convolutional layer.

Working of Grad-CAM

During testing

Step 1-Forward pass:

Input image

Convolutional layer 1

Convolutional layer 2

...

Last Convolutional layer

Kernels

Feature maps of last conv layer

A^1

A^2

A^3

A^4

$\vdots$

A^k

A logit is the value before applying the activation function at the output layer.

Logits → Softmax

A^k

There are k such feature maps in the last convolutional layer. Here $k = 5$

Flatten layer

Dense layers

y^c

c

Choose the class C (the predicted class with highest score). The value y^c is the logit (pre-softmax score) for class C.

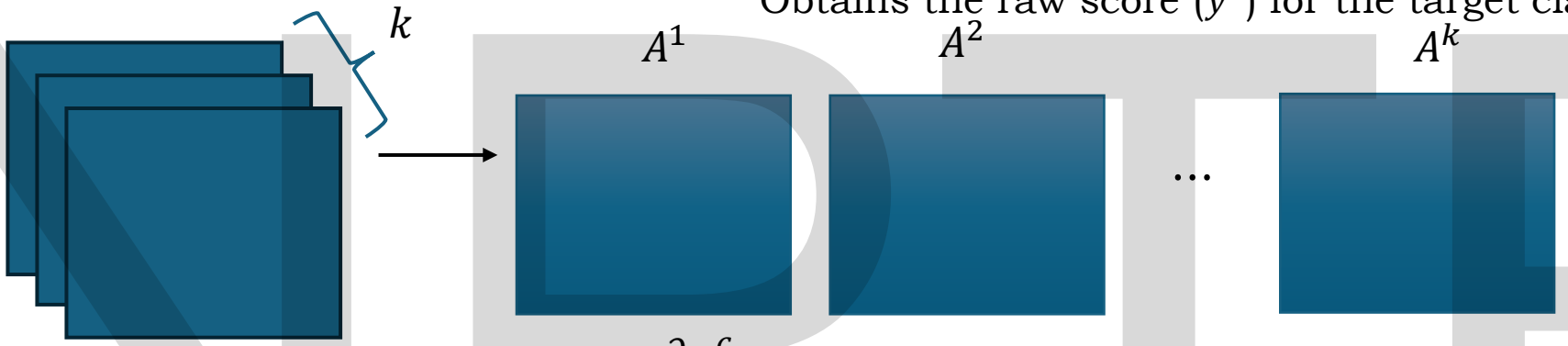
- Grad-CAM uses the **feature maps from the last conv layer** because it typically has the most high-level, semantically rich features.
- Earlier layers capture edges & textures (low-level features), but Grad-CAM needs high-level semantic features—hence it uses the last conv layer.

Pass the input image through the CNN to get the feature maps $A^1, A^2, \ldots, A^k$ from the last convolutional layer and raw output scores y^c (logits) before softmax.

Working of Grad-CAM

Step 2-Compute the gradients:

Forward Pass: Runs the image through the trained CNN.
Extracts the feature maps $(A^1, A^2, \dots, A^k)$ from the **final convolutional layer**.
Obtains the raw score (y^c) for the target class C.



Feature maps

Backward Pass: Performs one backward propagation from y^c.

•Computes the gradients of y^c **with respect to each activation** in the feature maps A^k.

y^c $\frac{\partial y^c}{\partial A^1_{i,j}}$ *gradient of the class score w.r.t each value of* A^1

A^1 *Feature map values*

2	1	...	...	...
1	0	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

0.2	0.1	...	...	...
0.1	-0.5	...	...	...
...	...	...	...	...
...	...	...	...	...
...	...	...	...	...

How much does changing location (i,j) of feature map A^1 affect the score of class C?

•If the gradient is **large and positive** across many locations in A^k
→this feature is **very important** for class C and thus gets larger weight a_k

•If gradients are small or negative
→ that feature doesn't help much for class C and gets smaller weight.

Average Influence of A^1

$$a_1 = \frac{1}{z}\sum_i \sum_j \frac{\partial y^c}{\partial A^1_{i,j}}$$

Weight for feature map A^1

Z = number of spatial locations = height × width of the feature map

How important is this feature map for the output class?

•If the gradient is **large and positive** across many locations in A^k
→this feature is **very important** for class C and thus gets larger weight a_k
•If gradients are small or negative
→ that feature doesn't help much for class C and gets smaller weight.

$A^1 =$

2	1	0	2	3
1	0	1	1	2
0	1	3	2	1
1	2	0	1	0
0	1	2	1	3

$A^2 =$

1	0	2	1	0
0	2	1	0	1
2	1	0	1	2
1	0	1	2	0
0	1	2	0	1

$A^3 =$

2	1	0	1	2
1	0	2	1	0
0	2	1	0	1
1	2	0	1	0
2	1	0	2	1

$$a_1 = \frac{1}{z}\sum_i \sum_j \frac{\partial y^c}{\partial A^1_{i,j}}$$

$\frac{\partial y^c}{\partial A^1_{i,j}} =$

0.2	0.1	0.0	0.2	0.3
0.1	-0.5	0.2	0.1	0.3
0.0	0.1	0.4	0.2	0.1
0.1	0.3	0.0	0.2	−0.1
0.0	0.2	0.3	0.1	0.4

$0.2 + 0.1 + 0.0 + 0.2 + 0.3 = \mathbf{0.8}$

$0.1 - 0.5 + 0.2 + 0.1 + 0.3 = \mathbf{0.2}$

$0.0 + 0.1 + 0.4 + 0.2 + 0.1 = \mathbf{0.8}$

$0.1 + 0.3 + 0.0 + 0.2 - 0.1 = \mathbf{0.5}$

$0.0 + 0.2 + 0.3 + 0.1 + 0.4 = \mathbf{1.0}$

Total gradient sum = 3.3

$z = 5 \times 5 = 25$

$a_1 = \frac{3.3}{25} = 0.132$

0.1	0.2	0.1	0.0	−0.1
0.3	0.2	0.1	0.0	−0.1
0.5	0.3	0.1	0.0	−0.2
0.4	0.2	0.0	−0.1	−0.2
0.3	0.1	−0.2	−0.3	−0.4

$a_2 = 0.052$

0.2	0.1	0.0	0.2	0.1
0.3	0.1	0.5	0.2	0.3
0.4	0.2	0.7	0.4	0.2
0.3	0.1	0.2	0.6	0.5
0.2	0.4	0.9	0.2	0.2

$a_3 = 0.3$

$a_1 = 0.132, a_2 = 0.052, a_3 = 0.3$

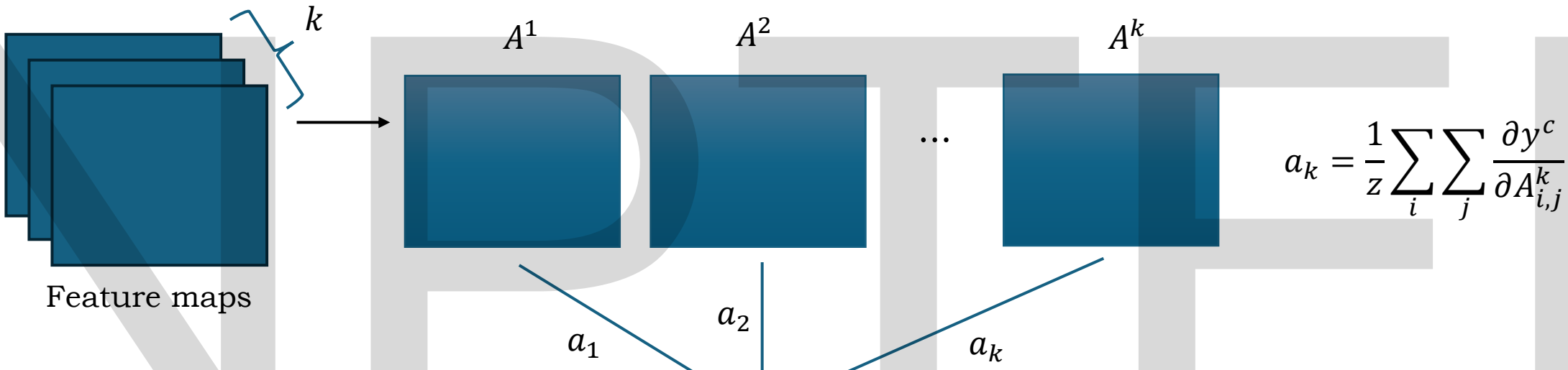
A' A² A³

A³ A' A²

Numerical example: Weight calculation

Working of Grad-CAM

Step 3-Heatmap creation:



$$a_k = \frac{1}{z}\sum_i \sum_j \frac{\partial y^c}{\partial A^k_{i,j}}$$

Multiply each feature map by its weight, then add them element-wise.

Grad-CAM Heatmap

0	0.9	...	...	...
0	0.6	0.87	...	...
0	0.55	0.97	0.89	...
0	...	...	...	...
0	...	...	...	...

ReLU
$f(x) = \max(0, x)$

-0.2	0.9	...	...	...
-0.1	0.6	0.87	...	...
-0.6	0.55	0.97	0.89	...
-0.7	...	...	...	...
-0.6	...	...	...	...

Weighted combination may contain negative values. We are only interested in the elements that have increased the logit of the predicted class. Apply ReLU

Localization Map (Weighted sum)

$$L = \sum_k a_k A^k$$

Grad-CAM Heatmap

$$L = ReLU\left(\sum_k a_k A^k\right)$$

A^1

2	1	0	2	3
1	0	1	1	2
0	1	3	2	1
1	2	0	1	0
0	1	2	1	3

A^2

1	0	2	1	0
0	2	1	0	1
2	1	0	1	2
1	0	1	2	0
0	1	2	0	1

A^3

2	1	0	1	2
1	0	2	1	0
0	2	1	0	1
1	2	0	1	0
2	1	0	2	1

$a_1 = 0.132$ $a_2 = 0.052$ $a_3 = 0.3$

Grad-CAM before ReLU

a_1A^1

0.264	0.132	0	0.264	0.396
0.132	0	0.132	0.132	0.264
0	0.132	0.396	0.264	0.132
0.132	0.264	0	0.132	0
0	0.132	0.264	0.132	0.396

a_2A^2

0.052	0	0.104	0.052	0
0	0.104	0.052	0	0.052
0.104	0.052	0	0.052	0.104
0.052	0	0.052	0.104	0
0	0.052	0.104	0	0.052

a_3A^3

0.6	0.3	0	0.3	0.6
0.3	0	0.6	0.3	0
0	0.6	0.3	0	0.3
0.3	0.6	0	0.3	0
0.6	0.3	0	0.6	0.3

$a_1A^1 + a_2A^2 + a_3A^3$

0.916	0.432	0.104	0.616	0.996
0.432	0.104	0.784	0.432	0.316
0.104	0.784	0.696	0.316	0.536
0.484	0.864	0.052	0.536	0.000
0.600	0.484	0.368	0.732	0.784

Numerical example: Heatmap creation

All values > 0 → positive contributions ✓
Very high values like 0.996, 0.916, 0.864 → strongest supporting regions
Smaller values like 0.052, 0.104 → weak evidence
Zero value → no contribution ✓

ReLU $f(x) = \max(0, x)$

ReLU removes all negative values if any, keeping only the positive contributions (the regions that support the predicted class).

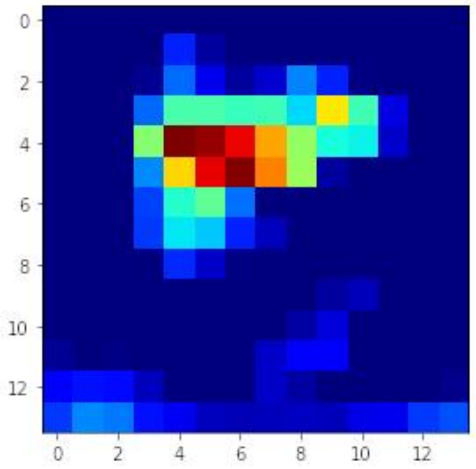
•CNNs end with **conv layers → then flatten → then dense layers → then softmax**
•After flattening, spatial structure is lost
→ Grad-CAM needs **spatial** feature maps
•Only convolutional layers have **2D spatial information.** Therefore, the **last convolutional layer** is the best layer that still retains semantic spatial information.

Step 4 – Upsample and overlay on Image:

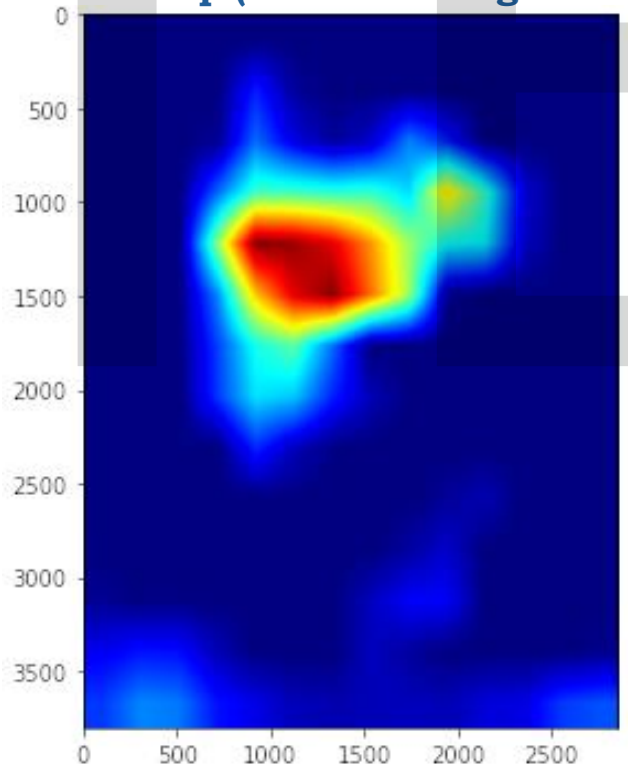
The Grad-CAM heatmap is much smaller than the input image because the final conv layer feature map is much smaller than the input image. Resize (upsample) the heatmap to the same size as the input so it can be overlaid on the original image. After resizing, overlay the heatmap on the original input. The resulting visualization highlights *which regions of the image were most important* for the model's prediction of class C. This shows **where the network 'looked'** when making its decision.

Original Image

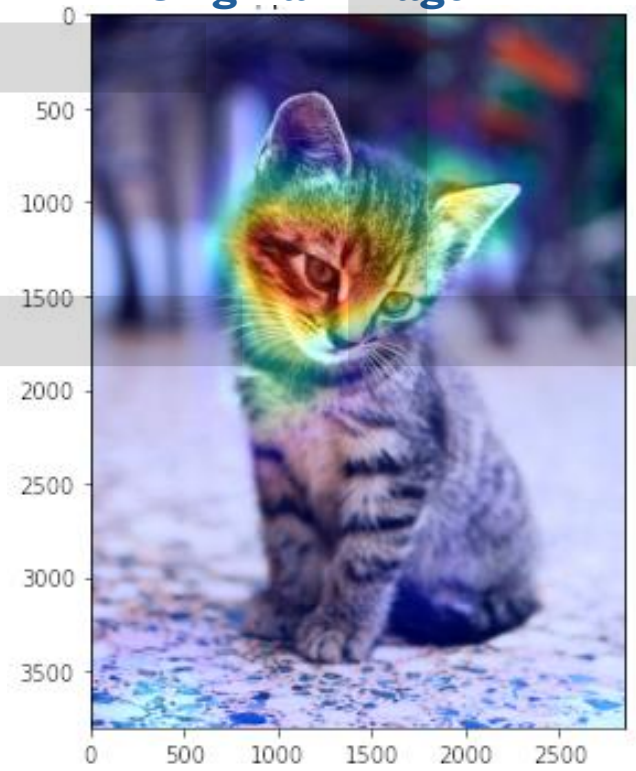
Grad-CAM Heatmap



Upsampled (Resized) Grad-CAM Heatmap (matches image size)



Grad-CAM Heatmap + Original Image



Working of CAMs

During testing

Step 1-Forward pass:

Input image

Convolutional layer 1

Convolutional layer 2

...

Last Convolutional layer

Kernels

Feature maps of last conv layer

$$GAP(A^k) = \frac{1}{H * W} \sum_{i=1}^{H} \sum_{j=1}^{W} A_{i,j}^k$$

Global Average Pooling (GAP)

Flatten layer

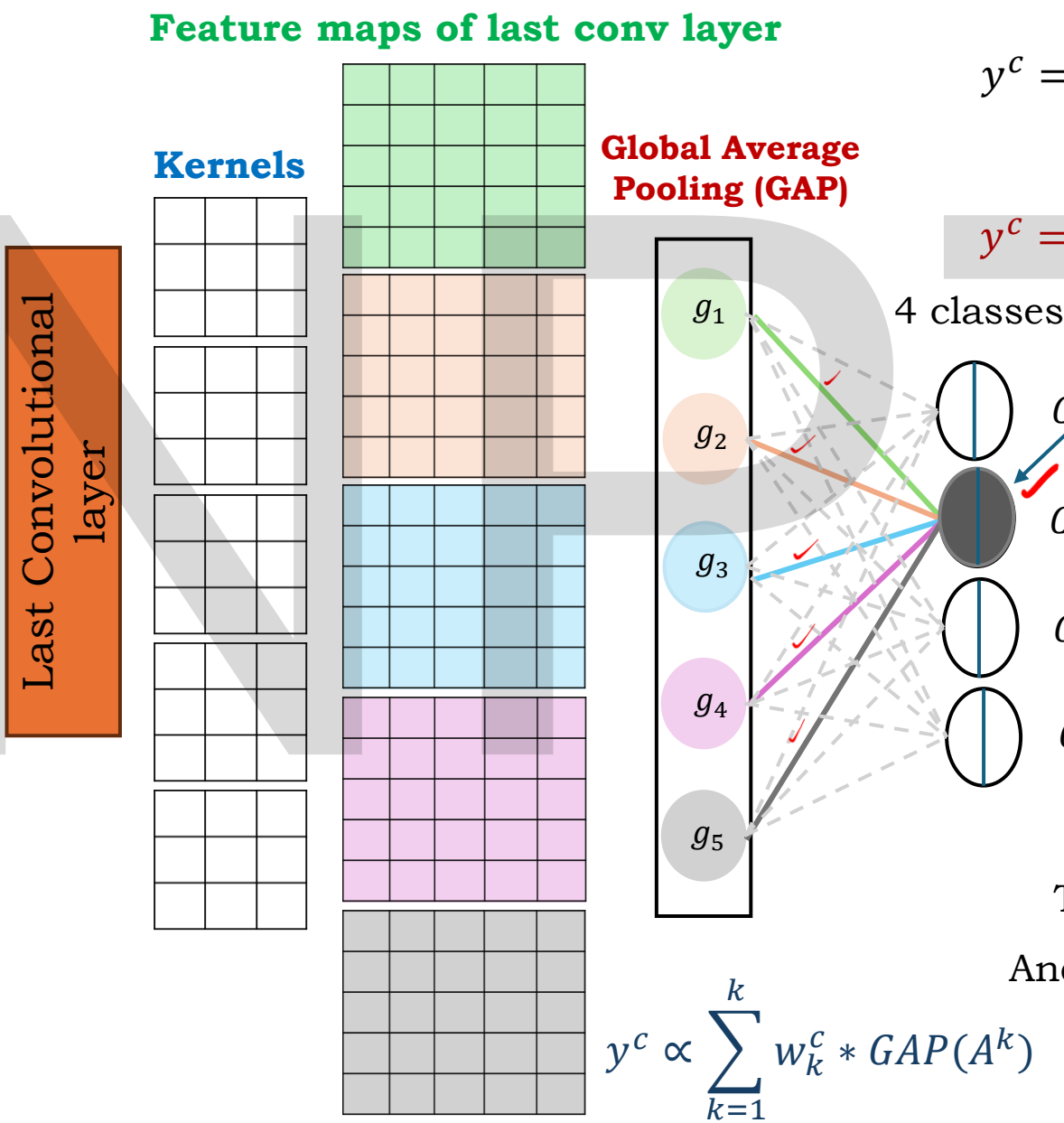
Suppose 4 classes

CAM (Class Activation Mapping) is XAI method that highlights which regions of an image were important for predicting a particular class.

But CAM works only for a special type of CNN architecture: Conv layers, GAP, Single fully connected layer.
This is *very restrictive.*

If a Flatten layer is used, all spatial information is mixed and lost. So, we can no longer know which region of the image influenced the prediction. This is why CAM needs a change.

Working of CAMs



$$y^c = \sum_{k=1}^{k} w_k^c * GAP(A^k) + b \quad GAP(A^k) = \frac{1}{H * W} \sum_{i=1}^{H} \sum_{j=1}^{W} A_{i,j}^k$$

$$y^c = \sum_{k=1}^{k} w_k^c \left(\frac{1}{H * W} \sum_{i=1}^{H} \sum_{j=1}^{W} A_{i,j}^k \right) = \frac{1}{H * W} \sum_{i=1}^{H} \sum_{j=1}^{W} \sum_{k=1}^{k} w_k^c A_{i,j}^k + b$$

$Class\ 2 \rightarrow y^c$

$Class\ 1: y^1 = w_1^1 g_1 + w_2^1 g_2 + w_3^1 g_3 + w_4^1 g_4 + w_5^1 g_5 + b$

$Class\ 2: y^2 = w_1^2 g_1 + w_2^2 g_2 + w_3^2 g_3 + w_4^2 g_4 + w_5^2 g_5 + b$

$Class\ 3: y^3 = w_1^3 g_1 + w_2^3 g_2 + w_3^3 g_3 + w_4^3 g_4 + w_5^3 g_5 + b$

$Class\ 4: y^4 = w_1^4 g_1 + w_2^4 g_2 + w_3^4 g_3 + w_4^4 g_4 + w_5^4 g_5 + b$

$$\hat{c} = argmax\{y^1, y^2, y^3, y^4\}$$

This class's logit becomes the **target logit y^c.**

And CAM will use **only that class's weights**:$w_1^c, w_2^c, w_3^c, w_4^c, w_5^c$

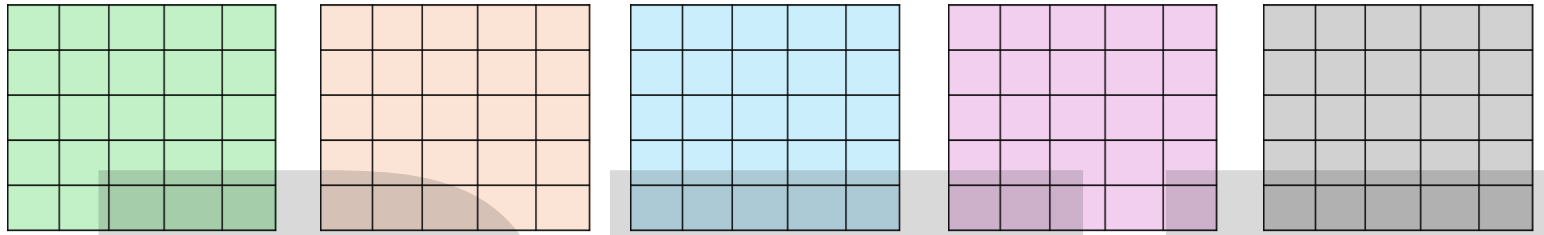
$$y^c \propto \sum_{k=1}^{k} w_k^c * GAP(A^k)$$

This shows y^c depends **directly on the activations inside each feature map.**

How CAM Produces a Single H × W Heatmap

Step 1 — Last Conv Layer gives **K feature maps**, each of size **H × W**

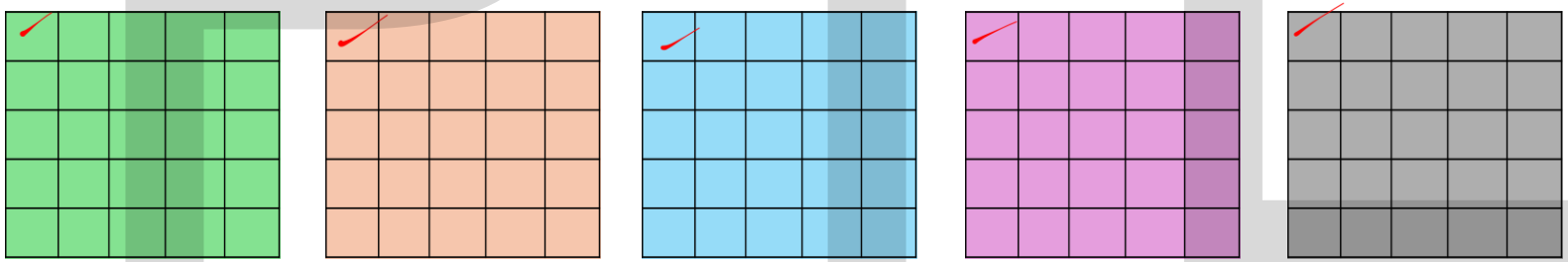
Example (K=5):



A^1 A^2 A^3 A^4 A^5

Step 2 — For the predicted class C, take its FC weights: $w_1^c, w_2^c, w_3^c, w_4^c, w_5^c$

Step 3 — Multiply each weight with the corresponding feature map $w_1^c \cdot A^1, w_2^c \cdot A^2, w_3^c \cdot A^3, w_4^c \cdot A^4, w_5^c \cdot A^5$

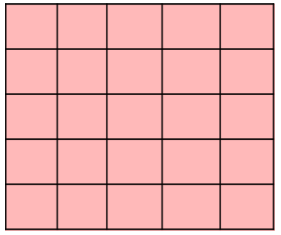


$w_1^c \cdot A^1$ $w_2^c \cdot A^2$ $w_3^c \cdot A^3$ $w_4^c \cdot A^4$ $w_5^c \cdot A^5$

Step 4 — Perform **element-wise addition** across all weighted maps

$$CAM^c = w_1^c \cdot A^1 + w_2^c \cdot A^2 + w_3^c \cdot A^3 + w_4^c \cdot A^4 + w_5^c \cdot A^5$$

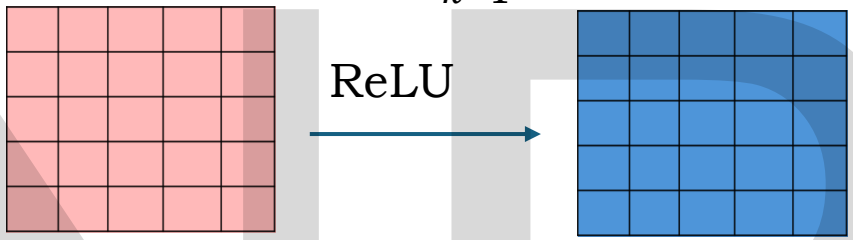
$$= \sum_{k=1}^{k} w_k^c \cdot A_{i,j}^k$$



How CAM Produces a Single H × W Heatmap

Step 5 — Apply ReLU

$$CAM^c = ReLU(\sum_{k=1}^{k} w_k^c \cdot A_{i,j}^k)$$



CAM heatmap before ReLU

CAM heatmap after ReLU

Higher CAM values → high importance

Lower CAM values → low importance

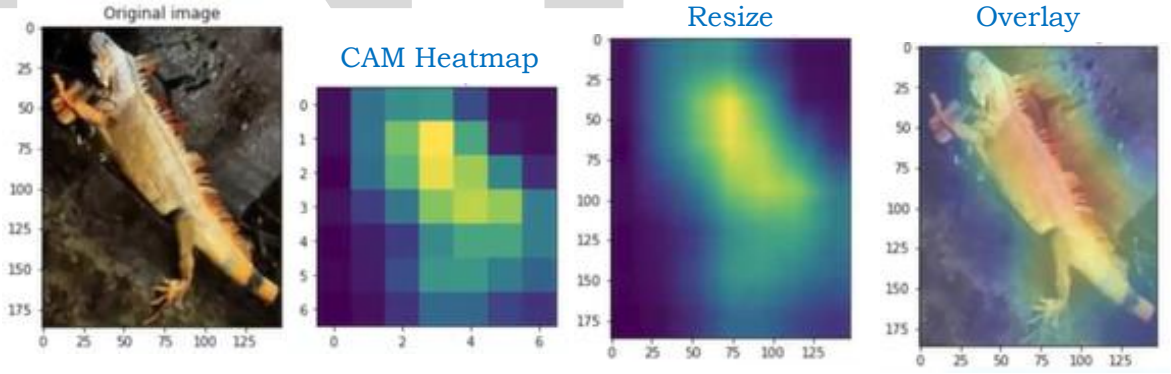
Zero or negative (if no ReLU) → no contribution

But CAM works only for a special type of CNN architecture. This is *very restrictive.*

Most modern CNNs (ResNet, VGG, EfficientNet) do not follow this architecture. This is why CAM cannot be used on many real CNN models. And this led to the invention of Grad-CAM.

Step 6 — Upsample Heatmap to Input Image Size and overlay Heatmap

CAM heatmap output is **H×W**. Usually much smaller than input because the final conv layer feature map is much smaller than the input image. So, resize (upsample/interpolate) to match the original image. After resizing, overlay the heatmap on the original input.



CAM is a **pure forward-pass technique.**

It uses only:

- Feature maps from last conv layer
- GAP values (one per feature map)
- Weights connecting Global Average Pooling (GAP) → final output neuron for the predicted class

No gradients are computed.

No backpropagation is performed.

Next Session

- We will see the hands-on implementation of the XAI methods

Thank you